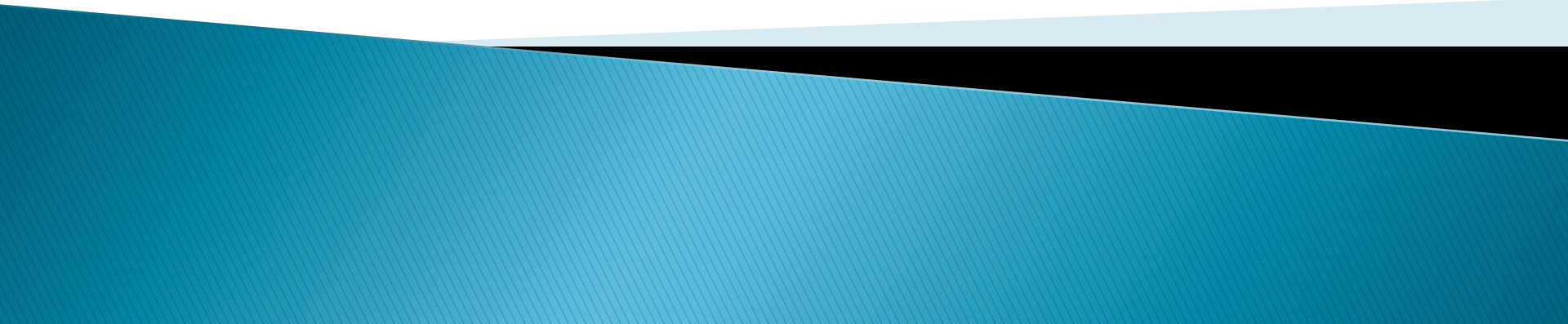


Алгоритмы сортировки данных



Сортировка

- ▶ процесс упорядочения заданного множества объектов по заданному признаку

Порядок сортировки

- ▶ по возрастанию: каждый следующий элемент больше предыдущего

$$a[1] < a[2] < \dots < a[n]$$

- ▶ по неубыванию: каждый следующий элемент не меньше предыдущего

$$a[1] \leq a[2] \leq \dots \leq a[n]$$

- ▶ по убыванию: каждый следующий элемент меньше предыдущего

$$a[1] > a[2] > \dots > a[n]$$

- ▶ по невозрастанию: каждый следующий элемент не больше предыдущего

$$a[1] \geq a[2] \geq \dots \geq a[n]$$

Алгоритмы сортировки

- ▶ методом простого выбора
- ▶ методом простого обмена
- ▶ методом прямого включения
(простыми вставками)
- ▶ слияниями
- ▶ быстрая сортировка (Хоара)
- ▶ пирамидальная сортировка
- ▶ ...

Сортировка методом простого выбора

Сортировка методом простого выбора

- ▶ выбрать максимальный элемент массива
- ▶ поменять его местами с последним элементом
- ▶ повторить предыдущие 2 пункта с оставшимися $n - 1$ элементами (рассмотреть часть массива, начиная с первого элемента до предпоследнего, найти в ней максимальный элемент и поменять его местами с предпоследним ($n - 1$)-ым элементом), затем с оставшимися $n - 2$ элементами и т.д., пока не останется один самый маленький элемент, уже стоящий на своем месте

Сортировка методом простого выбора

For $i = n-1$ Downto 1:

Begin

найти максимальный элемент

из $a[0], \dots, a[i]$

запомнить его индекс в переменной k

если $i \neq k$ поменять местами $a[i]$ и $a[k]$

End;

Сортировка методом простого выбора

```
For i = n-1 DownTo 1:  
    k = i  
    m = a[i]  
    For j = 0 To i-1:  
        if a[j] > m:  
            k = j  
            m = a[k]  
    If k != i:  
        a[k] = a[i]  
        a[i] = m
```

Сортировка методом простого выбора

процедура **sortSimpleSelection**

{Вход: массив $a[0..n-1]$.}

{Выход: массив a , отсортированный в порядке возрастания элементов.}

для i от $n-1$ до 1:

$maxi = i$

для j от 0 до $i-1$:

если $a[j] > a[maxi]$: $maxi = j$

если $maxi \neq i$: $a[i] \leftrightarrow a[maxi]$

Количество сравнений

- ▶ При первом просмотре $N - 1$, при втором – $N - 2$, при последнем – 1

- ▶ Общее количество

$$C = N - 1 + N - 2 + \dots + 1 = N * (N - 1) / 2$$

или $C = O(N^2)$

Модификация

- ▶ Метод предложен Э. Х. Фрэндом в 1956 году, получил название метода квадратичного выбора
- ▶ Количество сравнений имеет порядок $O(N * \text{Sqrt}(N))$
- ▶ N должно быть точным квадратом натурального числа. Массив делится на $\text{SQRT}(N)$ групп по $\text{SQRT}(N)$ элементов в каждой, в каждой группе выбирается максимальный элемент
- ▶ Пример
 - (7 10 3)(5 15 9)(6 12 8)
 - максимальные элементы 10 15 12
 - максимальный из них 15, он во второй группе. Если оставшиеся элементы из второй группы, а это 5 и 9, меньше 10 и 12, то мы нашли сразу три элемента, записываемые на свои места. Если нет, то заменяем максимальные элементы элементами из этой группы

Сортировка методом простого обмена

Сортировка методом простого обмена

- ▶ Метод заключается в последовательных просмотрах массива сверху вниз (от начала к концу) и обмене местами соседних элементов, расположенных “неправильно”, то есть таких, что $i < j$, а $a[i] > a[j]$

Сортировка методом простого обмена

процедура **sortSimpleExchange**

{Вход: массив $a[0..n-1]$.}

{Выход: массив a , отсортированный в порядке возрастания элементов.}

для i от 0 до $n-2$:

для j от 0 до $n-i-2$:

если $a[j] > a[j+1]$:

$a[j] \leftrightarrow a[j+1]$

Сортировка методом простого обмена

```
For k = 0 To n-2:  
  For i = 0 To n-k-2:  
    If a[i]>a[i+1]:  
      t = a[i]  
      a[i] = a[i+1]  
      a[i+1] = t
```

Анализ алгоритма

- ▶ При сортировке методом «пузырька» выполняется $N - 1$ просмотров, на каждом i -том просмотре производится $N - i$ сравнений
- ▶ Общее количество $C = N * (N - 1)/2$, или $O(N^2)$

Модификации

- ▶ Запоминать, производились ли перестановки элементов в процессе некоторого прохода. Если их не было, то сортировку можно закончить.
- ▶ Если известен не только факт последнего обмена, но и его место, то понятно, что все пары соседних элементов, расположенные правее этого места, уже находятся в нужном порядке, поэтому просмотр можно закончить на этом индексе, а не продолжать до конца
- ▶ «Шейкер-сортировка» (реализовать с рекурсией):
 - если при одном из проходов не было выполнено ни одной перестановки, то сортировку следует прекратить
 - чередовать направления сортировки, пока массив не будет отсортирован

Сортировка простыми вставками

Сортировка простыми вставками

- ▶ На k -ом шаге считается, что часть массива, содержащая первые $k - 1$ элемент уже упорядочена, то есть $a[1] \leq a[2] \leq \dots \leq a[k - 1]$. Далее необходимо взять k -ый элемент и подобрать для него место в отсортированной части массива такое, чтобы после его вставки упорядоченность не нарушится, то есть надо найти такое j ($1 \leq j \leq k - 1$), что $a[j] \leq a[k] < a[j + 1]$. Затем надо вставить элемент $a[k]$ на найденное место

Сортировка простыми вставками

For i = 1 To n-1:

w = a[i]

**вставить w на подходящее место в
a[0], ..., a[i]**

Сортировка простыми вставками

процедура **sortSimpleInsert**

{Вход: массив $a[0..n-1]$.}

{Выход: массив a , отсортированный в порядке возрастания элементов.}

для i от 1 до $n-1$:

$w = a[i], j = i-1$

пока $j > 0$ и $w < a[j]$:

$a[j+1] = a[j], j--$

$a[j+1] = w$

Анализ алгоритма

- ▶ В зависимости от сортируемого массива количество сравнений будет от $N - 1$ до $N * (N - 1)/2$ или $C = O(N^2)$
- ▶ Для массива, например, 1 2 3 4 5 6 7 8 потребуется $n - 1$ сравнение (напомним, что мы сортируем в порядке возрастания), а для массива 8 7 6 5 4 3 2 1 имеем $n \cdot (n - 1)/2$ или $c = O(n^2)$

Модификации

- ▶ Убрать из процедуры сортировки переменную x , а внутренний цикл записать в виде
 $\text{While } A[0] < A[j] \text{ Do...}$ Массив A необходимо сделать типа $\text{Array}[0..N_{max}]$ и k -й элемент записывать на 0 место (так называемый прием «барьерный элемент»)
- ▶ На момент вставки элемента с номером i элементы массива с номерами от 1 до $i - 1$ отсортированы. Поиск места для вставляемого элемента можно выполнять с помощью бинарного поиска. Эта схема называется сортировкой бинарными вставками, она предложена Дж. Мочли в 1946 году

Сортировка по индексам

Сортировка по индексам

Задача сортировки без (непосредственных) перемещений элементов может быть решена путем построения индексного массива, при этом доступ к ключам элементов осуществляется только для того, чтобы выполнить операцию сравнения.

Предположим, что сортируемые элементы находятся в массиве $data[0], \dots, data[n-1]$ строкового типа и возможности перемещать их в силу тех или иных причин (возможно, из-за их значительных размеров) нет. Чтобы получить эффект сортировки, используется второй массив (массив a – индексов элементов). Необходима первоначальная инициализации $a[i]$ значениями i для $i=0, \dots, n-1$.

Сортировка по индексам

Другими словами, $a[0]$ получает значение индекса первого элемента данных, $a[1]$ – второго элемента данных и т.д. Целью сортировки является переупорядочение массива индексов таким образом, что $a[0]$ дает индекс элемента данных с наименьшим значением ключа, $a[1]$ – индекс элемента данных с минимальным значением ключа из числа оставшихся и т.д. Результат сортировки достигается за счет доступа к ключам через индексы – например, таким способом можно вывести массив в порядке выполненной сортировки.

Сортировка методом слияний

Сортировка методом слияний

- ▶ Один из первых в теории алгоритмов сортировки (предложен Дж. фон Нейманом в 1945 году). В логике реализован один из фундаментальных принципов информатики – «разделяй и властвуй»
- ▶ Метод состоит в разбиении данного массива на несколько частей, которые сортируются по отдельности, и в последующем составлении из нескольких уже упорядоченных массивов искомого упорядоченного массива

Пример слияния двух массивов

Процедура sl(k, m, n):

 i = k+1, j = m+1, k = 0

 while (i<m) And (j<n):

 if a[i]<=b[j] :

 c[k] = a[i]

 i++

 else:

 c[k] = b[j]

 j++

 k++

 while (i<m) :

 c[k] = a[i]

 i++, k++

 while (j<n) :

 c[k] = b[j]

 j++, k++

Работа в одном массиве с использованием дополнительного массива D

Процедура SI(left, right – числа, A – массив):

```
i = left, m = left + (right - left) // 2,
```

```
j = m+1, t = 0
```

```
while (i<=m) And (j<= right):
```

```
    if A[i]<=A[j]:
```

```
        D[t] = A[i], i++
```

```
    else:
```

```
        D[t] = A[j], j++
```

```
    t++
```

```
while i<=m: D[t] = A[i], i++, t++
```

```
while j<= right : D[t] = A[j], j++, t++
```

```
for i = 0 To t-1: A[left + i] = D[i]
```

Процедура SortSl(i,j):

```
if (j-i=1) and (a[j]<a[i]):  
    a[i]<->a[j]  
else if i<j:  
    SortSl(i, i+(j-i)// 2)  
    SortSl(i+(j-i)// 2+1,j)  
    Sl(i, j, A)
```

- ▶ Первый вызов SortSl(0, n-1)

Анализ алгоритма

- ▶ Эффективность алгоритма составляет

$$C = O(N * \log N)$$